НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Звіт до комп'ютерного практикуму №4
З дисципліни «Комп'ютерне моделювання дискретно-подійних систем»
«Оцінка складності алгоритму імітації»

Прийняв:
Викладач
Коринецький Дмитро Сергійович
«04» листопада 2026 р.

Виконав:
Студент 4 курсу, гр. ІП-34
Скачков Іван
Олексійович

2026 р.

Зміст

Лабораторна робота №4 3
Технічне завдання 4
Виконання 5
Завдання 1 5
Лістинг файлів 6
Результат роботи 12
Висновок 13
Завдання 2 14
Лістинг файлів 14
Результат роботи 16
Висновок 17
Завдання 3 18
Висновок 19
Завдання 4 20
Лістинг файлів 21
Результат роботи 21
Висновок 23
Висновок 25
Посилання 26

Лабораторна робота №4

Тема: Оцінка складності алгоритму імітації.

Мета: навчитися оцінювати складність алгоритму імітації мережі масового обслуговування експериментально, за вимірюваннями часу обчислення, і теоретично, за кількістю подій та кількістю операцій на обробку однієї події; дослідити вплив структури маршруту на складність алгоритму.

Мова реалізації: Python 3.12, Matplotlib — лише для побудови графіків. За основу взято універсальний алгоритм імітації з комп'ютерного практикуму №3 (класи Element, Create, Process, Dispose, Model), до якого додано лише лічильник виконаних подій.

Технічне завдання

1. Розробити модель масового обслуговування, яка складається з N послідовно з’єднаних систем масового обслуговування. Число N є параметром моделі. Кількість подій в моделі оцінюється числом N + 1.
2. Виконати експериментальну оцінку складності алгоритму імітації мережі масового обслуговування. Для цього виконати серію експериментів, в якій спостерігається збільшення часу обчислення алгоритму імітації при збільшенні кількості подій в моделі.
3. Виконати теоретичну оцінку складності побудованого алгоритму імітації.
4. Повторити експериментальну та теоретичну оцінку складності алгоритму імітації при зміні структури мережі масового обслуговування: замість послідовно з’єднаних систем масового обслуговування розглянути розгалужені маршрути слідування між системами масового обслуговування. Зробити висновок про вплив розгалужень маршруту на складність алгоритму.

Виконання

Завдання 1

Модель складається з елемента CREATE, N послідовно з'єднаних одноканальних систем масового обслуговування P1, …, PN з необмеженими чергами та елемента DISPOSE (рисунок 1). Вимоги надходять з експоненційним інтервалом із середнім tC = 2, час обслуговування в кожній СМО експоненційний із середнім 1. Число N задається параметром: мережа будується функцією chain(n) модуля networks.py, тому та сама програма без змін імітує мережу з будь-якою кількістю СМО.

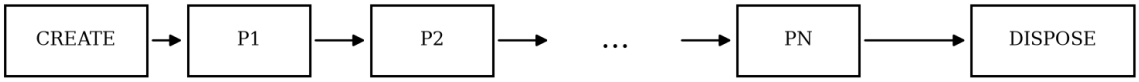


Рисунок 1 – Мережа з N послідовно з'єднаних СМО

Подіями моделі є надходження вимоги (подія елемента CREATE) і завершення обслуговування в кожній з N систем, тобто кількість подій у моделі дорівнює N + 1. Вихід вимоги з моделі окремою подією не є: він відбувається в момент завершення обслуговування в PN. Для оцінки складності до класу Model додано лічильник виконаних подій events. Інші класи взято з практикуму №3 без змін (таблиця 1).

Таблиця 1 – Модулі програми

Файл	Призначення
element.py, create.py, process.py, dispose.py, fun_rand.py	Класи універсального алгоритму з практикуму №3 без змін: вимога і базовий елемент, джерело, система обслуговування, вихід, генератори випадкових величин
model.py	Алгоритм імітації за принципом найближчої події з практикуму №3 і лічильник виконаних подій
networks.py	Побудова мереж з N СМО: послідовної та трьох розгалужених; кожна функція повертає модель і середню кількість СМО на маршруті вимоги
complexity.py	Вимірювання часу імітації, теоретична оцінка, графіки
task1_chain.py	Завдання 1: мережа з N послідовних СМО і її перевірка
task2_complexity.py	Завдання 2–3: експериментальна і теоретична оцінка складності
task4_branching.py	Завдання 4: оцінка складності для розгалужених маршрутів

Лістинг файлів

Файл fun_rand.py:

```
"""Генератори випадкових величин (аналог класу FunRand)."""
import math
import random


_rng = random.Random()



def seed(value):
    _rng.seed(value)



def uniform01():
    return _rng.random()



def exp(mean):
    return -mean * math.log(1 - _rng.random())



def unif(low, high):
    return low + _rng.random() * (high - low)



def norm(mean, dev):
    return mean + dev * _rng.gauss(0, 1)



def erlang(mean, k):
    """Сума k експоненційних величин із сумарним середнім mean."""
    return -mean / k * sum(math.log(1 - _rng.random()) for _ in range(k))
```

Файл element.py:

```
"""Вимога та базовий елемент моделі: часова затримка і вибір наступного елемента."""
import math


import fun_rand



class Customer:
    def __init__(self, kind, t_start):
        self.kind = kind  # поточний тип, може змінюватись у моделі
        self.origin = kind  # тип при надходженні
        self.t_start = t_start



class Element:
    def __init__(self, name, delay_mean=1.0, delay_dev=0.0, distribution="exp"):
        self.name = name
        self.delay_mean = delay_mean
        self.delay_dev = delay_dev
        self.distribution = distribution  # назва закону або функція(вимога) -> затримка
        self.tcurr = 0.0
```

```
        self.tnext = math.inf
        self.quantity = 0
        self.routes = []
        self.routing = "probability"
        self.last_in = None
        self.gap_sum = 0.0
        self.gaps = 0


    def get_delay(self, customer=None):
        if callable(self.distribution):
            return self.distribution(customer)
        if self.distribution == "exp":
            return fun_rand.exp(self.delay_mean)
        if self.distribution == "norm":
            return fun_rand.norm(self.delay_mean, self.delay_dev)
        if self.distribution == "unif":
            return fun_rand.unif(self.delay_mean, self.delay_dev)
        if self.distribution == "erlang":
            return fun_rand.erlang(self.delay_mean, self.delay_dev)
        return self.delay_mean


    def set_next(self, element):
        self.set_routes_by_probability([(element, 1.0)])


    def set_routes_by_probability(self, routes):
        """routes — список (елемент, ймовірність)."""
        self.routes, self.routing = routes, "probability"


    def set_routes_by_priority(self, elements):
        """elements — елементи в порядку спадання пріоритету."""
        self.routes, self.routing = elements, "priority"


    def set_router(self, function):
        """function(вимога) -> елемент: довільне правило вибору маршруту."""
        self.routing = function


    def next_element(self, customer):
        if callable(self.routing):
            return self.routing(customer)
        if self.routing == "priority":
            for element in self.routes:  # перший за пріоритетом, який може прийняти вимогу
                if element.can_accept():
                    return element
            return self.routes[0] if self.routes else None
        r, cumulative = fun_rand.uniform01(), 0.0
        for element, probability in self.routes:
            cumulative += probability
            if r < cumulative:
                return element
        return None


    def can_accept(self):
        return True


    def mark_arrival(self):
        if self.last_in is not None:
            self.gap_sum += self.tcurr - self.last_in
            self.gaps += 1
        self.last_in = self.tcurr


    def mean_interval(self):
        """Середній інтервал між надходженнями вимог в елемент."""
        return self.gap_sum / self.gaps if self.gaps else 0.0


    def in_act(self, customer):
        pass


    def out_act(self):
        self.quantity += 1


    def do_statistics(self, delta):
```

```
        pass

    def print_info(self):
        print(f"  {self.name}: quantity = {self.quantity}, tnext = {self.tnext:.4f}")


    def print_result(self, time):
        print(f"{self.name}: quantity = {self.quantity}")
```

Файл create.py:

```
"""CREATE: джерело вимог."""
from element import Customer, Element



class Create(Element):
    def __init__(self, name, delay_mean, delay_dev=0.0, distribution="exp", kind=None):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.tnext = 0.0
        self.kind = kind  # функція, що повертає тип нової вимоги


    def out_act(self):
        super().out_act()
        self.tnext = self.tcurr + self.get_delay()
        customer = Customer(self.kind() if self.kind else 0, self.tcurr)
        target = self.next_element(customer)
        if target:
            target.in_act(customer)
```

Файл process.py:

```
"""PROCESS: багатоканальне обслуговування з обмеженою чергою і пріоритетом у черзі."""
import math


from element import Element



class Process(Element):
    def __init__(self, name, delay_mean=1.0, delay_dev=0.0, distribution="exp",
                 channels=1, max_queue=math.inf, queue_key=None):
        super().__init__(name, delay_mean, delay_dev, distribution)
        self.channels = channels  # math.inf — затримка без обмеження кількості пристроїв
        self.max_queue = max_queue
        self.queue_key = queue_key  # функція(вимога) -> пріоритет у черзі, менше — раніше
        self.in_service = []  # пари (момент завершення, вимога)
        self.queue = []
        self.arrived = 0
        self.failure = 0
        self.queue_area = 0.0
        self.busy_area = 0.0


    @property
    def busy(self):
        return len(self.in_service)


    def can_accept(self):
        return self.busy < self.channels or len(self.queue) < self.max_queue


    def start_service(self, customer, delay=None):
        delay = self.get_delay(customer) if delay is None else delay
        self.in_service.append((self.tcurr + delay, customer))
        self.tnext = min(t for t, _ in self.in_service)


    def in_act(self, customer):
        self.mark_arrival()
        self.arrived += 1
        if self.busy < self.channels:
            self.start_service(customer)
```

```
        elif len(self.queue) < self.max_queue:
            self.queue.append(customer)
        else:
            self.failure += 1


    def take_from_queue(self):
        if self.queue_key is None:
            return self.queue.pop(0)
        index = min(range(len(self.queue)), key=lambda i: (self.queue_key(self.queue[i]), i))
        return self.queue.pop(index)


    def out_act(self):
        for item in [x for x in self.in_service if x[0] == self.tcurr]:
            self.in_service.remove(item)
            super().out_act()
            if self.queue:
                self.start_service(self.take_from_queue())
            self.tnext = min((t for t, _ in self.in_service), default=math.inf)
            customer = item[1]
            target = self.next_element(customer)
            if target:
                target.in_act(customer)


    def do_statistics(self, delta):
        self.queue_area += len(self.queue) * delta
        self.busy_area += self.busy * delta


    def mean_queue(self, time):
        return self.queue_area / time


    def mean_load(self, time):
        """Частка зайнятих пристроїв; для необмеженої кількості — середнє число зайнятих."""
        return self.busy_area / time / (1 if self.channels == math.inf else self.channels)


    def failure_probability(self):
        return self.failure / self.arrived if self.arrived else 0.0


    def print_info(self):
        print(f"  {self.name}: busy = {self.busy}/{self.channels}, queue = {len(self.queue)}, "
              f"quantity = {self.quantity}, failure = {self.failure}, tnext = {self.tnext:.4f}")


    def print_result(self, time):
        print(f"{self.name}: arrived = {self.arrived}, quantity = {self.quantity}, "
              f"failure = {self.failure}")
        print(f"    mean queue = {self.mean_queue(time):.4f}, "
              f"mean load = {self.mean_load(time):.4f}, "
              f"failure probability = {self.failure_probability():.4f}")
```

Файл dispose.py:

```
"""DESPOSE: вихід вимог із моделі та час їх перебування в системі."""
from element import Element



class Dispose(Element):
    def __init__(self, name):
        super().__init__(name)
        self.times = {}  # тип вимоги -> [кількість, сумарний час у системі]


    def in_act(self, customer):
        self.mark_arrival()
        self.quantity += 1
        stat = self.times.setdefault(customer.origin, [0, 0.0])
        stat[0] += 1
        stat[1] += self.tcurr - customer.t_start


    def mean_time(self, kind=None):
        """Середній час у системі для вимог типу kind (None — для всіх)."""
        stats = [s for k, s in self.times.items() if kind is None or k == kind]
```

```
        count = sum(s[0] for s in stats)
        return sum(s[1] for s in stats) / count if count else 0.0
```

Файл model.py:

```
"""Model: універсальний алгоритм імітації за принципом найближчої події."""



class Model:
    def __init__(self, elements, after_event=None):
        self.elements = elements
        self.after_event = after_event  # правило конкретної моделі, виконується після кожної події
        self.tcurr = 0.0
        self.events = 0  # кількість виконаних подій


    def simulate(self, time, verbose_events=0):
        """verbose_events — скільки перших подій вивести у протокол."""
        step = 0
        while True:
            tnext = min(e.tnext for e in self.elements)
            if tnext > time:  # статистика збирається рівно на інтервалі [0; time]
                tnext = time
            for e in self.elements:
                e.do_statistics(tnext - self.tcurr)
            self.tcurr = tnext
            if self.tcurr >= time:
                break
            for e in self.elements:
                e.tcurr = self.tcurr
            for e in self.elements:
                if e.tnext == self.tcurr:
                    if step < verbose_events:
                        print(f"\nПодія в {e.name}, t = {self.tcurr:.4f}")
                    e.out_act()
                    self.events += 1
            if self.after_event:
                self.after_event()
            if step < verbose_events:
                for e in self.elements:
                    e.print_info()
            step += 1


    def print_result(self):
        print(f"\n------------ РЕЗУЛЬТАТИ (T = {self.tcurr:g}) ------------")
        for e in self.elements:
            e.print_result(self.tcurr)
```

Файл networks.py:

```
"""Мережі з N СМО: послідовна і розгалужені структури маршруту.


Кожна функція повертає модель і середню кількість СМО, які проходить одна вимога.
"""
from create import Create
from dispose import Dispose
from model import Model
from process import Process


ARRIVAL, SERVICE, P_RETURN = 2.0, 1.0, 0.3



def _elements(n):
    create = Create("CREATE", ARRIVAL)
    processes = [Process(f"P{i}", SERVICE) for i in range(1, n + 1)]
    return create, processes, Dispose("DISPOSE")



def chain(n):
    """CREATE -> P1 -> P2 -> ... -> Pn -> DISPOSE: вимога проходить усі n СМО."""
```

```
    create, processes, dispose = _elements(n)
    for source, target in zip([create, *processes], [*processes, dispose]):
        source.set_next(target)
    return Model([create, *processes, dispose]), n



def tree(n):
    """Бінарне дерево: Pi передає вимогу в P2i або P2i+1 з імовірністю 0,5; з листя — DISPOSE."""
    create, processes, dispose = _elements(n)
    create.set_next(processes[0])
    for i, process in enumerate(processes, 1):
        children = [processes[j - 1] if j <= n else dispose for j in (2 * i, 2 * i + 1)]
        process.set_routes_by_probability([(child, 0.5) for child in children])
    visits = sum(0.5 ** (i.bit_length() - 1) for i in range(1, n + 1))  # Pi на глибині log2(i)
    return Model([create, *processes, dispose]), visits



def fan(n):
    """Віяло: CREATE передає вимогу в одну з n паралельних СМО з імовірністю 1/n."""
    create, processes, dispose = _elements(n)
    create.set_routes_by_probability([(process, 1 / n) for process in processes])
    for process in processes:
        process.set_next(dispose)
    return Model([create, *processes, dispose]), 1



def loop(n):
    """Ланцюг із поверненням: після Pn вимога з імовірністю P_RETURN повертається в P1."""
    model, _ = chain(n)
    processes, dispose = model.elements[1:-1], model.elements[-1]
    processes[-1].set_routes_by_probability([(processes[0], P_RETURN),
                                             (dispose, 1 - P_RETURN)])
    return model, n / (1 - P_RETURN)



STRUCTURES = {"послідовна": chain, "дерево": tree, "віяло": fan, "з поверненням": loop}
```

Файл task1_chain.py:

```
"""Завдання 1: модель з N послідовно з'єднаних СМО, N — параметр моделі."""
import fun_rand
from networks import ARRIVAL, SERVICE, chain


N, TIME = 5, 100_000



def main():
    fun_rand.seed(1)
    model, _ = chain(N)
    model.simulate(TIME, verbose_events=3)
    model.print_result()


    create, processes, dispose = model.elements[0], model.elements[1:-1], model.elements[-1]
    rho = SERVICE / ARRIVAL
    in_system = sum(len(p.queue) + p.busy for p in processes)
    balance = create.quantity == dispose.quantity + in_system
    print(f"\nКількість подій у моделі: N + 1 = {N + 1} (надходження і завершення "
          f"обслуговування в кожній з {N} СМО)")
    print(f"Виконано подій: {model.events}, очікується ν·T = (N + 1)·T / tC = "
          f"{(N + 1) * TIME / ARRIVAL:.0f}")
    print(f"Теорія (кожна СМО — M/M/1 з необмеженою чергою): завантаження {rho:.4f}, "
          f"середня черга {rho ** 2 / (1 - rho):.4f}")
    print(f"Баланс: створено {create.quantity} = вийшло {dispose.quantity} "
          f"+ у системі {in_system}: {'так' if balance else 'НІ'}")



if __name__ == "__main__":
```

```
main()
```

Результат роботи

Програму запущено для N = 5 на інтервалі T = 100000. Протокол перших подій і результати наведено нижче.

```
Подія в CREATE, t = 0.0000
  CREATE: quantity = 1, tnext = 0.2886
  P1: busy = 1/1, queue = 0, quantity = 0, failure = 0, tnext = 1.4430
  P2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P4: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P5: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DISPOSE: quantity = 0, tnext = inf


Подія в CREATE, t = 0.2886
  CREATE: quantity = 2, tnext = 0.8775
  P1: busy = 1/1, queue = 1, quantity = 0, failure = 0, tnext = 1.4430
  P2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P4: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P5: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DISPOSE: quantity = 0, tnext = inf


Подія в CREATE, t = 0.8775
  CREATE: quantity = 3, tnext = 2.0713
  P1: busy = 1/1, queue = 2, quantity = 0, failure = 0, tnext = 1.4430
  P2: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P3: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P4: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  P5: busy = 0/1, queue = 0, quantity = 0, failure = 0, tnext = inf
  DISPOSE: quantity = 0, tnext = inf


------------ РЕЗУЛЬТАТИ (T = 100000) -------------
CREATE: quantity = 49816
P1: arrived = 49816, quantity = 49815, failure = 0
    mean queue = 0.4954, mean load = 0.4974, failure probability = 0.0000
P2: arrived = 49815, quantity = 49814, failure = 0
    mean queue = 0.5030, mean load = 0.5002, failure probability = 0.0000
P3: arrived = 49814, quantity = 49814, failure = 0
    mean queue = 0.5064, mean load = 0.4996, failure probability = 0.0000
P4: arrived = 49814, quantity = 49813, failure = 0
    mean queue = 0.5059, mean load = 0.4974, failure probability = 0.0000
P5: arrived = 49813, quantity = 49813, failure = 0
    mean queue = 0.5065, mean load = 0.4976, failure probability = 0.0000
DISPOSE: quantity = 49813


Кількість подій у моделі: N + 1 = 6 (надходження і завершення обслуговування в кожній з 5 СМО)
Виконано подій: 298885, очікується ν·T = (N + 1)·T / tC = 300000
Теорія (кожна СМО — M/M/1 з необмеженою чергою): завантаження 0.5000, середня черга 0.5000
Баланс: створено 49816 = вийшло 49813 + у системі 3: так
```

Таблиця 2 – Перевірка мережі з N = 5 послідовних СМО (T = 100000)

СМО	Надійшло	Обслужено	Завантаження	Теорія	Середня черга	Теорія
P1	49816	49815	0,4974	0,5000	0,4954	0,5000
P2	49815	49814	0,5002	0,5000	0,5030	0,5000
P3	49814	49814	0,4996	0,5000	0,5064	0,5000
P4	49814	49813	0,4974	0,5000	0,5059	0,5000
P5	49813	49813	0,4976	0,5000	0,5065	0,5000

Висновок

Модель з N послідовних СМО побудовано так, що кількість систем є параметром. При пуассонівському потоці і необмежених чергах кожна СМО мережі за теоремою Джексона поводиться як система M/M/1 з навантаженням $\rho = 1/2 = 0{,}5$, для якої середнє завантаження дорівнює 0,5, а середня черга — $\rho^2/(1 - \rho) = 0{,}5$. Результати імітації збігаються з цими значеннями для всіх п’яти систем (таблиця 2).

За час моделювання виконано 298885 подій при очікуваних $\nu \cdot T = (N + 1) \cdot T/tC = 300000$: кожна з 49816 вимог породжує $N + 1 = 6$ подій. Баланс виконується: створено 49816 вимог, вийшло 49813, у системі залишилось 3. Модель працює правильно і може використовуватись для дослідження складності алгоритму.

Завдання 2

Складність алгоритму оцінено експериментально: при незмінному модельному часі T = 1000 кількість СМО збільшувалась удвічі — N = 15, 30, 60, 120, тобто кількість подій у моделі зростала від 16 до 121. Для кожного N вимірювався час обчислення алгоритму імітації (функція time.perf_counter). Щоб вимірювання відображали саме складність алгоритму, прийнято такі заходи.

- Перед вимірюванням модель «прогрівається» протягом модельного часу 10·N, за який вимоги встигають пройти всю мережу. Без цього в довгому ланцюжку останні СМО довго залишаються порожніми, і подій виконується менше, ніж у стаціонарному режимі.
- На час вимірювання вимикається збирач сміття Python, щоб він не вносив випадкових затримок.
- Короткі прогони повторюються, доки сумарний час не досягне 0,3 с, і береться найменший час, як це робиться в модулі timeit.
- Генератор випадкових чисел ініціалізується однаково, тому кількість подій у кожному прогоні відтворювана; час обчислення від запуску до запуску змінюється на кілька відсотків.

Лістинг файлів

Файл complexity.py:

```
"""Експериментальна і теоретична оцінка складності алгоритму імітації мережі з N СМО."""
import gc
import math
import statistics
import time as clock
from pathlib import Path


import fun_rand
from networks import ARRIVAL


TIME = 1000  # модельний час, на якому вимірюється час обчислення
NS = (15, 30, 60, 120)  # кожне наступне N удвічі більше
OUT = Path(__file__).parent / "output"



def measure(build, n, budget=0.3):
    """Рядок (N, час обчислення в с, виконано подій, надійшло вимог, V).


    V — середня кількість СМО, які проходить вимога: кожна вимога породжує 1 + V подій.
    Короткі прогони повторюються, доки сумарний час не досягне budget, і береться найменший час.
    """
```

```
    best, spent = math.inf, 0.0
    while spent < budget:
        fun_rand.seed(1)
        model, visits = build(n)
        warmup = 10 * n  # за цей час вимоги проходять мережу — у всіх СМО стаціонарний режим
        model.simulate(warmup)
        create = model.elements[0]
        events, arrivals = model.events, create.quantity
        gc.disable()  # збирач сміття не втручається у вимірювання
        start = clock.perf_counter()
        model.simulate(warmup + TIME)
        elapsed = clock.perf_counter() - start
        gc.enable()
        best, spent = min(best, elapsed), spent + elapsed
    return n, best, model.events - events, create.quantity - arrivals, visits



def series(build, ns=NS):
    return [measure(build, n) for n in ns]



def per_event(rows):
    """Час на одну подію як лінійна функція N: a + b·N, мкс (метод найменших квадратів)."""
    fit = statistics.linear_regression([r[0] for r in rows], [r[1] / r[2] * 1e6 for r in rows])
    return fit.intercept, fit.slope



def theory_time(n, visits, a, b):
    """ν·T·k: подій за час T, помножених на час обробки однієї події a + b·N."""
    return TIME / ARRIVAL * (1 + visits) * (a + b * n) / 1e6



def order(ns, values):
    """Показник m у залежності «значення ~ N^m»: нахил прямої в логарифмічному масштабі."""
    return statistics.linear_regression([math.log(n) for n in ns],
                                        [math.log(v) for v in values]).slope



def print_series(name, rows, a, b):
    print(f"\nСтруктура: {name}")
    print("    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія "
          "| зріс у | на подію, мкс")
    previous = None
    for n, elapsed, events, arrivals, visits in rows:
        growth = f"{elapsed / previous:6.2f}" if previous else "     —"
        print(f"{n:5} | {n + 1:14} | {events:14} | {events / arrivals:6.2f} / {1 + visits:<6.2f} | "
              f"{elapsed:7.3f} / {theory_time(n, visits, a, b):<7.3f}       | {growth} | "
              f"{elapsed / events * 1e6:8.2f}")
        previous = elapsed
    ns = [r[0] for r in rows]
    print(f"Час обчислення ~ N^{order(ns, [r[1] for r in rows]):.2f} (імітація), "
          f"~ N^{order(ns, [theory_time(r[0], r[4], a, b) for r in rows]):.2f} (теорія)")



def plot(results, a, b, filename, log=False):
    """results — назва структури -> (функція побудови мережі, рядки вимірювань)."""
    import matplotlib.pyplot as plt


    fig, (left, right) = plt.subplots(1, 2, figsize=(11, 4.2))
    dense = range(NS[0], NS[-1] + 1)
    for name, (build, rows) in results.items():
        ns = [r[0] for r in rows]
        line, = left.plot(ns, [r[1] for r in rows], "o", label=f"{name}: імітація")
        left.plot(dense, [theory_time(n, build(n)[1], a, b) for n in dense], "--",
                  color=line.get_color(), label=f"{name}: теорія")
        right.plot(ns, [r[1] / r[2] * 1e6 for r in rows], "o-", label=name)
    right.plot(dense, [a + b * n for n in dense], "k--", label=f"{a:.2f} + {b:.3f}·N")
    if log:  # у логарифмічному масштабі залежність N^m — пряма з нахилом m
        left.set(xscale="log", yscale="log")
        left.set_xticks(NS, labels=[str(n) for n in NS])
```

```
        left.minorticks_off()
    left.set(xlabel="N — кількість СМО", ylabel="час обчислення, с",
             title=f"Час імітації на інтервалі T = {TIME}")
    right.set(xlabel="N — кількість СМО", ylabel="час на одну подію, мкс",
              title="Час обробки однієї події")
    for ax in (left, right):
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8)
    fig.tight_layout()
    OUT.mkdir(exist_ok=True)
    fig.savefig(OUT / filename, dpi=110)
    print(f"\nГрафік збережено: output/{filename}")
```

Файл task2_complexity.py:

```
"""Завдання 2-3: експериментальна і теоретична оцінка складності для послідовної мережі."""
from complexity import TIME, per_event, plot, print_series, series
from networks import ARRIVAL, chain



def main():
    rows = series(chain)
    a, b = per_event(rows)
    print(f"Модельний час T = {TIME}, середній інтервал надходження tC = {ARRIVAL}")
    print_series("послідовна", rows, a, b)


    print("\nТеоретична оцінка: O(ν·T·k)")
    print("  ν = (N + 1) / tC — подій за одиницю часу: надходження + завершення в кожній з N СМО;")
    print("  k — операцій на одну подію: 4 проходи по списку з N + 2 елементів (мінімальний tnext,")
    print("      статистика, розсилання tcurr, перевірка tnext) + обробка самої події;")
    print(f"      виміряно: час на подію ≈ {a:.2f} + {b:.3f}·N мкс — лінійно зростає з N;")
    print("  ν·T·k = T / tC · (N + 1) · (a + b·N)  =>  O(N²): при подвоєнні N час зростає")
    print("      майже вчетверо (до 4 разів при великих N).")
    plot({"послідовна": (chain, rows)}, a, b, "task2_chain.png")



if __name__ == "__main__":
    main()
```

Результат роботи

```
Модельний час T = 1000, середній інтервал надходження tC = 2.0


Структура: послідовна
    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія | зріс у | на подію, мкс
   15 |             16 |           8507 |  16.08 / 16.00  |   0.061 / 0.052           |      — |     7.16
   30 |             31 |          14737 |  31.49 / 31.00  |   0.160 / 0.169           |   2.63 |    10.87
   60 |             61 |          32380 |  62.15 / 61.00  |   0.598 / 0.596           |   3.74 |    18.48
  120 |            121 |          60118 | 118.34 / 121.00 |   2.246 / 2.233           |   3.76 |    37.37
Час обчислення ~ N^1.75 (імітація), ~ N^1.81 (теорія)


Теоретична оцінка: O(ν·T·k)
  ν = (N + 1) / tC — подій за одиницю часу: надходження + завершення в кожній з N СМО;
  k — операцій на одну подію: 4 проходи по списку з N + 2 елементів (мінімальний tnext,
      статистика, розсилання tcurr, перевірка tnext) + обробка самої події;
      виміряно: час на подію ≈ 2.20 + 0.289·N мкс — лінійно зростає з N;
  ν·T·k = T / tC · (N + 1) · (a + b·N)  =>  O(N²): при подвоєнні N час зростає
      майже вчетверо (до 4 разів при великих N).


Графік збережено: output/task2_chain.png
```

Таблиця 3 – Час обчислення для послідовної мережі (T = 1000)

N	Подій у моделі	Виконано подій	Подій на одну вимогу	Час, с	Зростання при подвоєнні N	Час на подію, мкс
15	16	8507	16,08	0,061	—	7,16
30	31	14737	31,49	0,160	2,63	10,87
60	61	32380	62,15	0,598	3,74	18,48
120	121	60118	118,34	2,246	3,76	37,37

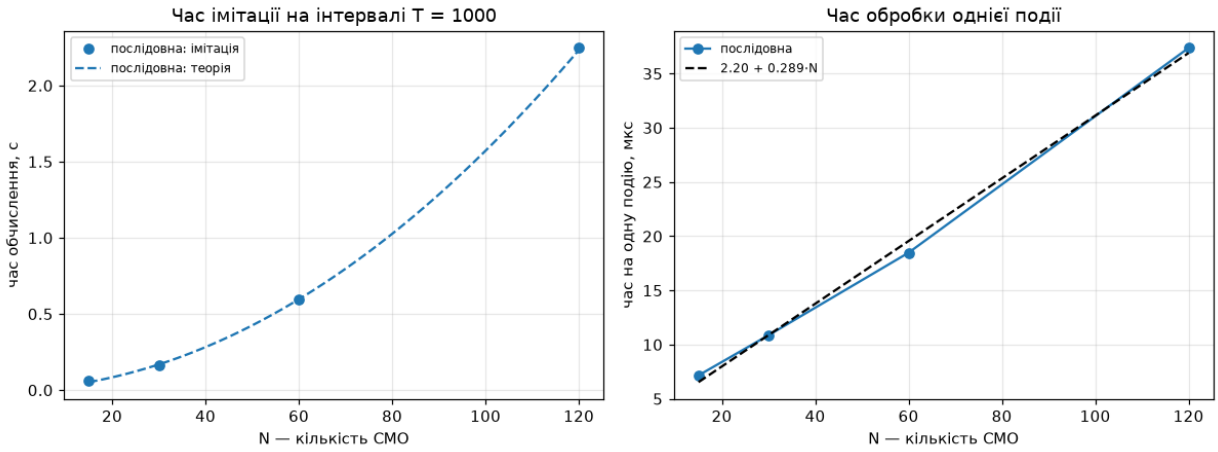


Рисунок 2 – Час обчислення і час обробки однієї події для послідовної мережі

Висновок

Час обчислення зростає значно швидше, ніж кількість подій у моделі: при збільшенні N у 8 разів (з 15 до 120) він збільшився з 0,061 до 2,246 с, тобто в 37 разів. При кожному подвоєнні N час зростає в 2,63, 3,74 і 3,76 раза, наближаючись до 4, що відповідає квадратичній залежності.

Таке зростання складається з двох множників. По-перше, кількість виконаних подій зростає пропорційно N: кожна вимога породжує N + 1 подію (стовпець «Подій на одну вимогу»). По-друге, час обробки однієї події також лінійно зростає з N — від 7,16 до 37,37 мкс (рисунок 2, праворуч), бо на кожному кроці алгоритм переглядає всі елементи моделі. Апроксимація у логарифмічному масштабі дає залежність часу обчислення ~N^1,75; показник менший за 2, оскільки при малих N помітну частку часу займає обробка самої події, яка від N не залежить.

Завдання 3

Теоретична оцінка складності алгоритму імітації має вигляд $O(\nu \cdot T \cdot k)$, де ν — інтенсивність подій (кількість подій за одиницю модельного часу), T — час моделювання, k — максимальна кількість елементарних операцій на обробку однієї події. Добуток $\nu \cdot T$ дає кількість подій за час моделювання, а множення на k — загальну кількість операцій.

Інтенсивність подій. Вимоги надходять в середньому через tC одиниць часу. Кожна вимога породжує подію надходження і по одній події завершення обслуговування в кожній СМО свого маршруту. У послідовній мережі вимога проходить усі N систем, тому в стаціонарному режимі $\nu = (N + 1)/tC$. Втрат немає, бо черги необмежені.

Кількість операцій на подію. Один крок алгоритму Model.simulate (таблиця 4) складається з чотирьох проходів по списку з N + 2 елементів (CREATE, N систем, DISPOSE) та обробки самої події. Обробка події — обчислення наступного моменту, генерація випадкового числа, передача вимоги наступному елементу і вибір маршруту — від N не залежить і дає сталу c.

Таблиця 4 – Операції одного кроку алгоритму імітації

Дія алгоритму	Кількість операцій
Пошук найменшого моменту tnext серед елементів	N + 2
Накопичення статистики do_statistics за інтервал до наступної події	N + 2
Розсилання поточного часу tcurr елементам	N + 2
Перевірка tnext = tcurr і виконання події out_act → in_act	N + 2 + c
Разом на одну подію, k	4(N + 2) + c

Отже, загальна кількість операцій за час моделювання:

$$\nu \cdot T \cdot k = (T / tC) \cdot (N + 1) \cdot (4(N + 2) + c) = O(T \cdot N^2 / tC).$$

Складність алгоритму для послідовної мережі квадратична за кількістю СМО (і за кількістю подій у моделі N + 1) та лінійна за часом моделювання. При подвоєнні N кількість подій і кількість операцій на одну подію подвоюються, тому при великих N час обчислення зростає вчетверо.

Порівняння з експериментом. Виміряний час на одну подію добре описується лінійною залежністю a + b·N з a = 2,20 мкс і b = 0,289 мкс (метод найменших квадратів), що підтверджує оцінку k = 4(N + 2) + c: на один елемент списку на кожному кроці витрачається близько 72 нс на кожен із чотирьох проходів, а обробка самої події займає кілька мікросекунд. Теоретичний час (T/tC)·(N + 1)·(a + b·N) наведено в таблиці 5 поруч з виміряним.

Таблиця 5 – Час обчислення для послідовної мережі: імітація і теорія

N	Подій на вимогу: імітація	Теорія N + 1	Час, с: імітація	Теорія, с
15	16,08	16	0,061	0,052
30	31,49	31	0,160	0,169
60	62,15	61	0,598	0,596
120	118,34	121	2,246	2,233

Висновок

Теоретична оцінка O(ν·T·k) з ν = (N + 1)/tC і k = 4(N + 2) + c дає квадратичну складність $O(N^2)$. Вона узгоджується з експериментом: кількість подій на одну вимогу збігається з N + 1, теоретичний час відрізняється від виміряного не більше ніж на кілька відсотків (крім найменшого N, де час обчислення найменший і вимірювання найменш точне), а показник степеня на дослідженому діапазоні становить 1,75 в експерименті і 1,81 за теоретичною формулою. При збільшенні N стала c стає дедалі менш помітною порівняно з 4(N + 2), і показник наближається до 2.

Квадратична складність є наслідком того, що алгоритм на кожному кроці переглядає всі елементи моделі. Її можна знизити, зберігаючи моменти наступних подій у черзі з пріоритетом (купі) і накопичуючи статистику елемента лише тоді, коли змінюється його стан; тоді обробка події займає O(log N) операцій.

Завдання 4

Для дослідження впливу розгалужень маршруту розглянуто три структури мережі з тими самими N системами (рисунок 3); параметри надходження та обслуговування — такі самі, як у послідовній мережі.

- Дерево: вимога з CREATE надходить у P1, а після системи Pi з ймовірністю 0,5 переходить у P2i або P2i+1; із систем, що не мають наступних, вимога виходить з моделі. Вимога проходить лише одну гілку дерева — близько $\log_2(N + 1)$ систем.
- Віяло: CREATE спрямовує вимогу в одну з N паралельних систем з ймовірністю 1/N, після якої вимога виходить з моделі. Вимога проходить одну систему.
- З поверненням: послідовна мережа, у якій після PN вимога з ймовірністю p = 0,3 повертається в P1. Вимога проходить ланцюжок у середньому 1/(1 − p) разів, тобто N/(1 − p) систем.

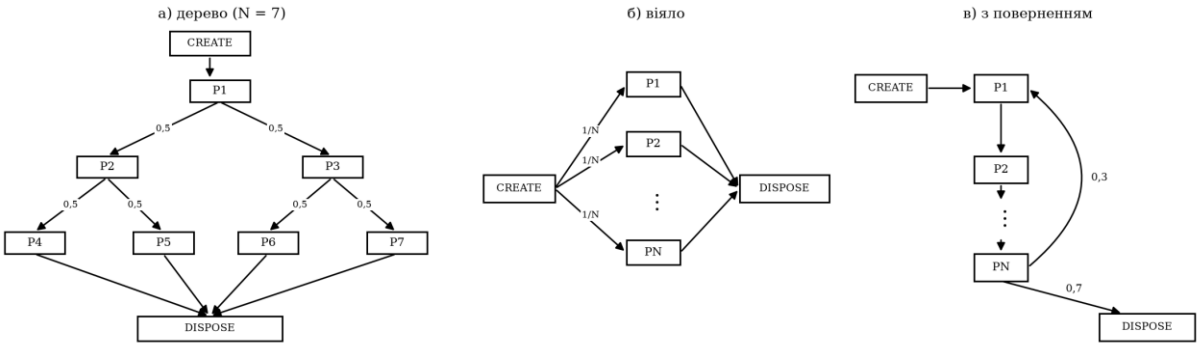


Рисунок 3 – Розгалужені структури мережі

Теоретична оцінка будується так само, як для послідовної мережі. Позначимо через V середню кількість систем, які проходить вимога; тоді вона породжує 1 + V подій, і $\nu = (1 + V)/tC$. Кількість операцій на подію $k = 4(N + 2) +$ c не залежить від структури, бо алгоритм переглядає всі елементи моделі незалежно від маршруту. Вибір маршруту за ймовірністю перебирає маршрути елемента: у дереві їх два, а у віялі CREATE перебирає до N маршрутів, що додає до k ще O(N) операцій і порядку складності не змінює. Отже, складність $O(\nu \cdot T \cdot k) = O(T \cdot (1 + V) \cdot N/tC)$ (таблиця 6).

Таблиця 6 – Теоретична оцінка складності для різних структур

Структура	V — СМО на маршруті	ν — подій за одиницю часу	Складність
послідовна	N	(N + 1)/tC	$O(N^2)$
дерево	$\approx \log_2(N + 1)$	$(1 + \log_2(N + 1))/tC$	$O(N \cdot \log N)$
віяло	1	2/tC	O(N)
з поверненням	$N/(1 - p)$	$(1 + N/(1 - p))/tC$	$O(N^2)$

Лістинг файлів

Функції побудови розгалужених мереж tree, fan і loop наведено в лістингу networks.py (завдання 1).

Файл task4_branching.py:

```
"""Завдання 4: складність алгоритму імітації для розгалужених маршрутів."""
from complexity import NS, TIME, order, per_event, plot, print_series, series, theory_time
from networks import ARRIVAL, P_RETURN, STRUCTURES


THEORY = {  # V — середня кількість СМО на маршруті вимоги; асимптотична складність
    "послідовна": ("N", "O(N²)"),
    "дерево": ("≈ log2(N + 1)", "O(N·log N)"),
    "віяло": ("1", "O(N)"),
    "з поверненням": (f"N / (1 - {P_RETURN})", "O(N²)"),
}



def main():
    print(f"Модельний час T = {TIME}, середній інтервал надходження tC = {ARRIVAL}")
    results = {name: series(build) for name, build in STRUCTURES.items()}
    a, b = per_event(sum(results.values(), []))  # алгоритм той самий — оцінка за всіма рядками
    for name, rows in results.items():
        print_series(name, rows, a, b)


    n = NS[-1]
    print(f"\nПідсумок для N = {n} (ν = (1 + V) / tC; час на подію ≈ {a:.2f} + {b:.3f}·N мкс):")
    print("  структура     | V              | подій  | час, с: імітація / теорія | ~N^m  "
          "| складність")
    for name, rows in results.items():
        visits, complexity = THEORY[name]
        _, elapsed, events, _, v = rows[-1]
        m = order(NS, [r[1] for r in rows])
        print(f"  {name:13} | {visits:14} | {events:6} | {elapsed:7.3f} / "
              f"{theory_time(n, v, a, b):<7.3f}         | {m:5.2f} | {complexity}")
    plot({name: (STRUCTURES[name], rows) for name, rows in results.items()}, a, b,
         "task4_branching.png", log=True)



if __name__ == "__main__":
    main()
```

Результат роботи

```
Модельний час T = 1000, середній інтервал надходження tC = 2.0


Структура: послідовна
    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія | зріс у | на подію, мкс
   15 |             16 |           8507 |  16.08 / 16.00  |   0.058 / 0.056           |      — |     6.84
```

```
   30 |             31 |          14737 |  31.49 / 31.00  |   0.151 / 0.170         |   2.60 |    10.25
   60 |             61 |          32380 |  62.15 / 61.00  |   0.622 / 0.581         |   4.12 |    19.22
  120 |            121 |          60118 | 118.34 / 121.00 |   2.149 / 2.126         |   3.45 |    35.75
Час обчислення ~ N^1.77 (імітація), ~ N^1.75 (теорія)


Структура: дерево
    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія | зріс у | на подію, мкс
   15 |             16 |           2582 |   5.00 / 5.00   |   0.018 / 0.017         |      — |     6.88
   30 |             31 |           2646 |   5.91 / 5.94   |   0.029 / 0.033         |   1.62 |    10.88
   60 |             61 |           3378 |   6.92 / 6.91   |   0.065 / 0.066         |   2.25 |    19.17
  120 |            121 |           3964 |   7.90 / 7.89   |   0.136 / 0.139         |   2.09 |    34.21
Час обчислення ~ N^1.00 (імітація), ~ N^1.00 (теорія)


Структура: віяло
    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія | зріс у | на подію, мкс
   15 |             16 |            953 |   2.00 / 2.00   |   0.006 / 0.007         |      — |     6.51
   30 |             31 |            958 |   2.00 / 2.00   |   0.010 / 0.011         |   1.60 |    10.36
   60 |             61 |            987 |   2.00 / 2.00   |   0.018 / 0.019         |   1.79 |    18.05
  120 |            121 |            984 |   2.00 / 2.00   |   0.033 / 0.035         |   1.88 |    34.03
Час обчислення ~ N^0.81 (імітація), ~ N^0.78 (теорія)


Структура: з поверненням
    N | подій у моделі | виконано подій | подій на вимогу | час, с: імітація / теорія | зріс у | на подію, мкс
   15 |             16 |          11676 |  22.50 / 22.43  |   0.091 / 0.078         |      — |     7.83
   30 |             31 |          20333 |  44.01 / 43.86  |   0.240 / 0.241         |   2.63 |    11.82
   60 |             61 |          39718 |  86.16 / 86.71  |   0.810 / 0.826         |   3.37 |    20.40
  120 |            121 |          82577 | 165.82 / 172.43 |   3.005 / 3.030         |   3.71 |    36.39
Час обчислення ~ N^1.69 (імітація), ~ N^1.76 (теорія)


Підсумок для N = 120 (ν = (1 + V) / tC; час на подію ≈ 2.94 + 0.268·N мкс):
  структура     | V              | подій  | час, с: імітація / теорія | ~N^m  | складність
  послідовна    | N              |  60118 |   2.149 / 2.126           |  1.77 | O(N²)
  дерево        | ≈ log2(N + 1)  |   3964 |   0.136 / 0.139           |  1.00 | O(N·log N)
  віяло         | 1              |    984 |   0.033 / 0.035           |  0.81 | O(N)
  з поверненням | N / (1 - 0.3)  |  82577 |   3.005 / 3.030           |  1.69 | O(N²)


Графік збережено: output/task4_branching.png
```

Таблиця 7 – Час обчислення для різних структур мережі, с (T = 1000)

Структура	N = 15	N = 30	N = 60	N = 120
послідовна	0,058	0,151	0,622	2,149
дерево	0,018	0,029	0,065	0,136
віяло	0,006	0,010	0,018	0,033
з поверненням	0,091	0,240	0,810	3,005

Таблиця 8 – Показники складності для різних структур (N = 120)

Структура	Подій на вимогу: імітація / 1 + V	Час, с: імітація / теорія	Показник m: імітація / теорія	Складність
послідовна	118,34 / 121,00	2,149 / 2,126	1,77 / 1,75	O(N²)
дерево	7,90 / 7,89	0,136 / 0,139	1,00 / 1,00	O(N·log N)
віяло	2,00 / 2,00	0,033 / 0,035	0,81 / 0,78	O(N)
з поверненням	165,82 / 172,43	3,005 / 3,030	1,69 / 1,76	O(N²)

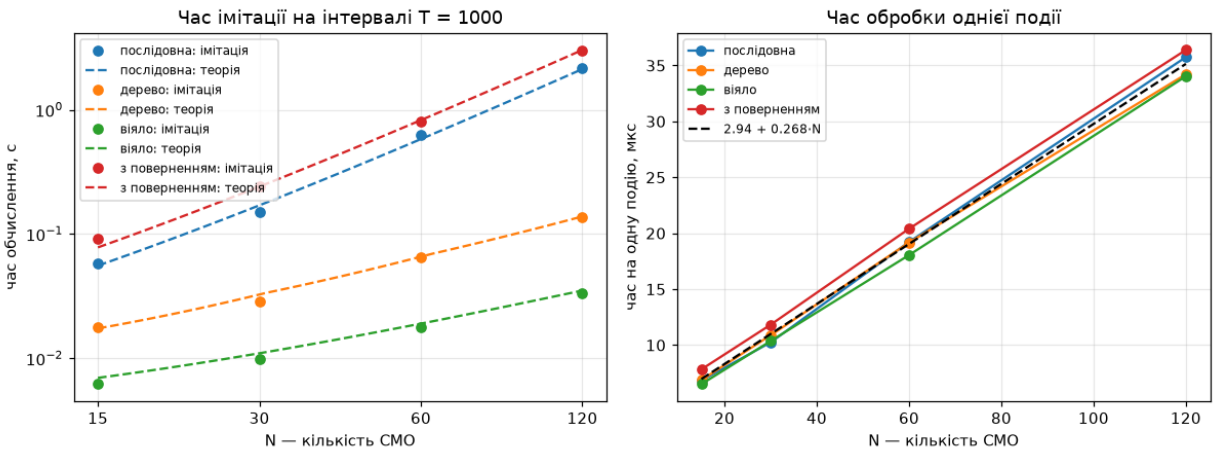


Рисунок 4 – Час обчислення (у логарифмічному масштабі) і час обробки однієї події для різних структур

Висновок

Час обробки однієї події однаковий для всіх структур (рисунок 4, праворуч) і лінійно зростає з N: за всіма вимірюваннями він становить 2,94 + 0,268·N мкс. Тому структура маршруту впливає на складність лише через інтенсивність подій ν, тобто через кількість систем, які проходить вимога. Кількість подій на одну вимогу в усіх випадках збігається з теоретичним значенням 1 + V, а теоретичний час — з виміряним.

1. Розгалуження без повернень зменшують складність, бо кожна вимога проходить лише одну гілку. У дереві $V \approx \log_2(N + 1)$: при N = 120 вимога породжує 7,90 події замість 118,34 у ланцюжку, а час обчислення в 16 разів менший; складність $O(N \cdot \log N)$. У віялі кожна вимога породжує 2 події незалежно від N, і час зростає лише через перегляд елементів: складність O(N), показник m = 0,81 на дослідженому діапазоні наближається до 1.
2. Розгалуження з поверненням збільшує кількість проходжень систем у 1/(1 − p) разів: при p = 0,3 час обчислення в 1,40 раза більший, ніж для послідовної мережі (теоретично в 1/0,7 ≈ 1,43 раза). Порядок складності залишається $O(N^2)$, але стала зростає, а при p, близькому до 1, кількість подій необмежено зростає.

Отже, розгалуження маршруту не змінюють кількості операцій на обробку однієї події, але змінюють кількість подій: складність алгоритму пропорційна середній довжині маршруту вимоги, помноженій на кількість елементів моделі.

Висновок

У ході виконання комп'ютерного практикуму оцінено складність універсального алгоритму імітації мереж масового обслуговування з практикуму №3.

1. Побудовано модель з N послідовно з'єднаних СМО, у якій N є параметром, а кількість подій дорівнює N + 1. Правильність моделі підтверджено збігом завантаження і довжини черг з теорією M/M/1 та балансом вимог.
2. Експериментально встановлено, що для послідовної мережі час обчислення при подвоєнні кількості СМО зростає майже вчетверо; на діапазоні N від 15 до 120 він пропорційний N^1,75.
3. Теоретична оцінка $O(\nu \cdot T \cdot k)$ з інтенсивністю подій $\nu = (N + 1)/tC$ і кількістю операцій на подію $k = 4(N + 2) + c$ дає квадратичну складність $O(N^2)$; теоретичний час узгоджується з виміряним.
4. Для розгалужених маршрутів кількість операцій на подію не змінюється, а кількість подій визначається середньою довжиною маршруту вимоги V. Розгалуження без повернень зменшують складність до $O(N \cdot \log N)$ для дерева і $O(N)$ для віяла; повернення збільшують час обчислення в $1/(1 - p)$ разів, не змінюючи порядку $O(N^2)$.

Складність алгоритму імітації визначається двома чинниками: скільки подій відбувається в моделі за час моделювання і скільки операцій потрібно на обробку однієї події. Перший чинник залежить від структури моделі, другий — від реалізації алгоритму.

Посилання

1. Комп’ютерний практикум 4 «Оцінка складності алгоритму імітації»: методичні вказівки з дисципліни «Комп’ютерне моделювання дискретно-подійних систем». – КПІ ім. Ігоря Сікорського, 2026.
2. Стеценко І. В. Моделювання систем: навч. посіб. – Черкаси: ЧДТУ, 2010. – 399 с.
3. Law A. M., Kelton W. D. Simulation Modeling and Analysis. – 3rd ed. – McGraw-Hill, 2000.
4. Повний код практичної роботи: https://github.com/ip34is/7-sem/tree/main/CMDES/CMDES-lab4